Covoiturage

Documentation Technique

Auteur : Obidah Hajjo

Version 1.0

Mars 2026

Table des matières

Documentation Technique 1
Table des matières 2
Introduction et présentation du projet 5
1.1 Objectifs fonctionnels 5
1.2 Environnement cible 5
Stack technique 6
2.1 Commandes disponibles 6
Structure des dossiers 7
Configuration et environnement 9
4.1 Résolution dynamique de l'URL de base 9
Architecture des providers — arbre React 10
5.1 LoadingProvider 10
5.2 AxiosInterceptorProvider 10
5.3 AuthProvider 11
5.4 ChatInboxProvider 11
Clients Http – Axios 12
6.1 apiClient — client interne (src/app/apiClient.ts) 12
6.2 externalApiClient — services tiers (src/app/externalApiClient.ts) 12
6.3 Gestion des erreurs API (src/app/apiError.ts) 13
Authentification et gestion de session 14
7.1 Stratégie d'authentification — JWT custom (firebase/php-jwt) 14
7.2 Payload JWT (access token) 14
7.3 Middleware LocalJwtAuth — Authentification des requêtes 14
7.4 Rotation des refresh tokens 15
7.5 Côté frontend — cycle de rafraîchissement transparent 15
7.6 Modèle AuthUser 16
7.7 Complétion de profil (src/auth/profileCompletion.ts) 16
Routage et gardes de routes 17
8.1 ProtectedRoute — logique de décision 17
8.2 PermissionRoute 17
8.3 Table complète des routes 18

Internationalisation (i18n) 19
9.1 Locales supportées 19
9.2 Résolution de la locale initiale 19
9.3 Fonction translate() 19
9.4 Hook useI18n et I18nProvider 20
9.5 Formatage des dates 20
Chat en temps réel — WebSocket 21
10.1 Initialisation du client Echo (chatEcho.ts) 21
10.2 useChatRealtime 21
10.3 Table des canaux WebSocket 22
10.4 État non-lu (chatReadState.ts) 22
10.5 useChatConversation — hook de conversation 23
Support en direct 24
11.1 Côté utilisateur — SupportChatWidget 24
11.2 Côté administrateur — Interface de gestion 24
Gestion du chargement global 25
12.1 Mécanisme de chargement global basé sur un compteur 25
Gestion des erreurs globales 26
13.1 Flash message de redirection 26
Modèles de données (Types TypeScript) 27
14.1 ApiResponse<T> — enveloppe standard 27
14.2 AuthUser et AuthPermissions 27
14.3 ChatConversation et ChatMessage 27
14.4 Trip 27
14.5 Address, Commune, GeoPfSearchResponse 28
Composants principaux 29
15.1 AppLayout (src/components/layout/AppLayout.tsx) 29
15.2 AdminLayout (src/components/layout/AdminLayout.tsx) 29
15.3 BottomNav (src/components/layout/BottomNav.tsx) 29
15.4 SerenePrimitives (src/components/common/SerenePrimitives.tsx) 29
15.5 FloatingToast (src/components/common/FloatingToast.tsx) 29
15.6 GlobalSpinner (src/components/common/GlobalSpinner.tsx) 29
15.7 LandingNavbar (src/components/common/LandingNavbar.tsx) 29

Conventions et patterns de développement 30
16.1 Pattern pages fines (thin pages) 30
16.2 Hooks propriétaires 30
16.3 Pas de gestion d'état globale externe 30
16.4 Gestion des erreurs dans les hooks 30
16.5 Styling — Tailwind + tokens CSS 30
16.6 FormData pour les pièces jointes 31
16.7 Mises à jour optimistes 31
Sécurité 32
17.1 Aucun token sensible en stockage client 32
17.2 Authentification des canaux WebSocket privés 32
17.3 Guards de routes côté frontend 32
17.4 Interception transparente des 401 32
17.5 Protection contre les clones d'objets (DataCloneError) 33

Introduction et présentation du projet

Covoit est une application web de covoiturage développée avec React 18 et TypeScript côté frontend, et Laravel côté backend. Elle permet à des conducteurs de proposer des trajets et à des passagers de les rechercher, réserver et contacter les conducteurs en temps réel via WebSocket.

Cette documentation technique décrit l'ensemble de l'architecture frontend : organisation du code, responsabilités de chaque couche, comportements complexes (authentification, WebSocket, i18n, gestion des erreurs) et conventions appliquées tout au long du projet.

1.1 Objectifs fonctionnels

- Inscription, connexion et gestion de profil utilisateur
- Recherche et réservation de trajets avec géocodage en temps réel (geo.api.gouv.fr)
- Publication et gestion de trajets conducteur
- Messagerie instantanée entre passagers et conducteurs (WebSocket + polling de secours)
- Support en direct avec un agent administrateur via chat
- Interface d'administration complète (utilisateurs, trajets, véhicules, support)
- Application multilingue : français, anglais, arabe (RTL automatique)
- Interface responsive — optimisée desktop et mobile

1.2 Environnement cible

Aspect	Valeur
Framework frontend	React 19 + TypeScript
Outil de build	Vite
Backend	Laravel (API REST + Reverb WebSocket)
Authentification	JWT custom (firebase/php-jwt HS256) + refresh tokens opaques (DB)
Port développement	127.0.0.1:5173

Stack technique

Catégorie	Technologie	Rôle
Framework UI	React 19	Composants fonctionnels, hooks, context API
Langage	TypeScript	Typage statique strict sur l'ensemble du projet
Build / Dev server	Vite 7	HMR, bundling optimisé, variables d'env VITE_
Styles	Tailwind CSS + PostCSS	Classes utilitaires, tokens CSS dans index.css
Routage	React Router v7	BrowserRouter, Outlet pattern, guards de route
HTTP client	Axios	apiClient (interne) + externalApiClient (tiers)
WebSocket	Laravel Echo + pusher-js	Transport Reverb, canaux privés authentifiés
i18n	Implémentation custom	Sans bibliothèque externe — translate(), dictionaries.ts
Backend API	Laravel (PHP)	REST API, Reverb WebSocket, JWT custom + refresh tokens DB
Géocodage communes	geo.api.gouv.fr	Recherche de communes françaises par nom
Géocodage GPS	data.geopf.fr	Adresse textuelle → coordonnées latitude/longitude

2.1 Commandes disponibles

```
npm run dev         # Serveur Vite → 127.0.0.1:5173
npm run build       # Vérification TypeScript + bundle production (dist/)
npm run lint        # ESLint sur tout le projet
npm run preview     # Serveur local du build production
```

Structure des dossiers

Le code source est organisé dans src/ selon une architecture en couches stricte. Chaque répertoire a une responsabilité unique, ce qui facilite la navigation, les tests et la maintenance.

```
src/
├── app/                Clients Axios et utilitaires d'erreur API
│   ├── apiClient.ts       Client interne (withCredentials, intercepteurs 401)
│   ├── externalApiClient.ts Client tiers sans cookie
│   └── apiError.ts        Extraction des messages d'erreur API
├── auth/
│   └── profileCompletion.ts  isProfileComplete(user)
├── bootstrap.ts        AxiosInterceptorProvider — montage des intercepteurs de chargement
├── components/
│   ├── common/         Primitives (FloatingToast, GlobalSpinner, SerenePrimitives…)
│   ├── layout/         Shells (AppLayout, AdminLayout, BottomNav, TopNav)
│   └── ui/             Sections UI par fonctionnalité
│       ├── Chat/       LiveChatSection, ChatInboxSection
│       ├── Support/    SupportSection, SupportChatWidget
│       ├── Admin/      Pages admin (Support, Users, Trips…)
│       └── …
├── context/            Contextes React (AuthContext, LoadingContext, ChatInboxContext)
├── features/           Fonctions d'appel API par domaine
│   ├── auth/           authApi.ts
│   ├── chat/           chatApi.ts, chatEcho.ts, chatReadState.ts, useChatRealtime.ts
│   ├── admin/          useAdminSupportChatApi.ts, useAdminSupportRealtime.ts
│   └── person/         personApi.ts
├── hooks/              Hooks par domaine
│   ├── Auth/           useAuth, useLogin, useRegister, useCompleteProfile
│   ├── Chat/           useChatInbox, useChatConversation
│   ├── Driver/         useMyTrips, usePublishTrip, useDriverTripDetails
│   ├── Booking/        useMyBookings, useBookingDetails
│   ├── Support/        useSupportCenter, useSupportChat
│   ├── Admin/          useAdminSupportInbox, useAdminSupportChat
│   └── …
├── i18n/
│   ├── config.ts       translate(), setCurrentLocale(), formatLocaleDateTime()
│   ├── dictionaries.ts Toutes les chaînes en EN / FR / AR
│   └── I18nProvider.tsx Hook useI18n, LanguageSwitcher
├── pages/              Composants de route (minimalistes — délèguent à ui/)
│   ├── Auth/           LoginPage, RegisterPage, ForgotPasswordPage, ResetPasswordPage
│   ├── Trips/          FindTripPage, TripResultsPage, TripDetailsPage, ContactDriver*
│   ├── DriverTrips/    MyTripsPage, PublishTripPage, DriverTripDetailsPage
│   ├── Bookings/       MyBookingsPage, BookingDetailsPage
│   ├── Chat/           ChatInboxPage, ChatConversationPage
│   ├── Admin/          AdminDashboardPage, AdminSupportInboxPage, AdminSupportChatPage…
│   └── …
├── providers/          AuthProvider, LoadingProvider, ChatInboxProvider
├── router/             AppRouter + guards (ProtectedRoute, PermissionRoute, AdminRoute…)
└── types/              Modèles TypeScript (MeResponse, Chat, Trip, ApiResponse…)
```

Couche	Chemin	Responsabilité
App infra	src/app/	Clients Axios, helpers d'erreur API
Providers	src/providers/	AuthProvider, LoadingProvider, ChatInboxProvider
Router	src/router/	Arbre de routes + composants guard
Features	src/features/	Fonctions d'appel API pures par domaine métier
Hooks	src/hooks/	Hooks page/domaine — données + état local
Context	src/context/	Contextes React consommés par hooks et providers
Pages	src/pages/	Composants de route — fins, délèguent à ui/
Components	src/components/	common/ (primitives), layout/ (shell), ui/ (sections)
Types	src/types/	Interfaces TypeScript pour objets API et domaine
i18n	src/i18n/	Système i18n custom — translate(), dictionnaires

Configuration et environnement

L'application utilise les variables d'environnement Vite (préfixe VITE_) lues au moment du build. Copier .env.example vers .env.local avant le premier démarrage.

```
VITE_API_BASE_URL=http://covoit.local    # URL de base du backend Laravel
VITE_REVERB_APP_KEY=...                  # Clé publique Laravel Reverb
VITE_REVERB_HOST=covoit.local            # Hôte WebSocket
VITE_REVERB_PORT=80
VITE_REVERB_WS_PORT=80
VITE_REVERB_WSS_PORT=443
VITE_REVERB_SCHEME=http                  # "http" ou "https"
```

4.1 Résolution dynamique de l'URL de base

La fonction resolveApiBaseUrl() dans src/app/apiClient.ts normalise l'URL configurée. Si l'API pointe vers localhost et que le frontend tourne également en localhost, elle remplace le hostname de l'URL de l'API par celui du navigateur actif. Cela garantit la cohérence quel que soit l'alias DNS utilisé en développement.

```
// src/app/apiClient.ts
function resolveApiBaseUrl(rawBaseUrl: string): string {
    try {
        const url = new URL(rawBaseUrl);
        const currentHostname = window.location.hostname;
        const isLocalApiHost = url.hostname === "localhost" || url.hostname
=== "127.0.0.1";
        const isLocalFrontendHost = currentHostname === "localhost" ||
currentHostname === "127.0.0.1";

        if (isLocalApiHost && isLocalFrontendHost) {
            url.hostname = currentHostname;
        }

        return url.toString().replace(/\/$/, "");
    } catch {
        return rawBaseUrl;
```

Architecture des providers — arbre React

Tous les providers globaux sont déclarés et imbriqués dans src/App.tsx selon un ordre précis qui respecte leurs dépendances.

```
// src/App.tsx
<LoadingProvider>                        // Compteur de requêtes actives
  <AxiosInterceptorProvider />           // Monte les intercepteurs Axios (effets de bord)
  <AuthProvider>                         // Session utilisateur + refreshMe
    <ChatInboxProvider>                  // Conversations inbox + badge non-lu
      <GlobalSpinner />                 // Overlay spinner (isLoading)
      <GlobalErrorAlert />              // Alertes d'erreur globales
      <AppRouter />                     // Arbre de routes
    </ChatInboxProvider>
  </AuthProvider>
</LoadingProvider>
```

Provider	Contexte exposé	Dépendance
LoadingProvider	pendingRequests, isLoading, startLoading, stopLoading	Aucune
AxiosInterceptorProvider	(effets de bord seulement — retourne null)	LoadingContext
AuthProvider	status, user, refreshMe, logoutLocal	apiClient
ChatInboxProvider	conversations, unreadCount, isRealtimeConnected, reload	AuthContext + Echo

5.1 LoadingProvider

Maintient un compteur d'entiers (pendingRequests). Chaque requête Axios active incrémente ce compteur via startLoading() ; chaque réponse (succès ou erreur) le décrémente via stopLoading() avec Math.max(0, prev-1) pour éviter les valeurs négatives. isLoading vaut true tant que pendingRequests > 0.

5.2 AxiosInterceptorProvider

Composant sans rendu (retourne null). Monte au montage quatre intercepteurs : un intercepteur de requête et un de réponse sur chacun des deux clients Axios (apiClient et externalApiClient). Les intercepteurs appellent startLoading() / stopLoading() sauf si config.showGlobalLoader === false — utilisé pour les appels de polling silencieux.

5.3 AuthProvider

Gère l'état d'authentification avec trois statuts possibles : "loading" (vérification en cours au démarrage), "authenticated" (session valide, objet user disponible), "guest" (non connecté). Au montage, appelle refreshMe() qui tente GET /auth/me . En cas d'échec 401, tente POST /auth/refresh (renouvelle le cookie), puis relance GET /auth/me. Si les deux échouent, statut → "guest".

```
// src/providers/AuthProvider.tsx
const refreshMe = async (): Promise<boolean> => {
  try {
    const me = await getMe();             // GET /auth/me
    setUser(me); setStatus("authenticated");
    return true;
  } catch {
    try {
      await apiClient.post("/auth/refresh"); // renouvelle le cookie
      const me = await getMe();
      setUser(me); setStatus("authenticated");
      return true;
    } catch {
      setUser(null); setStatus("guest");
      return false;
    }
  }
};
```

5.4 ChatInboxProvider

S'abonne au canal WebSocket inbox de l'utilisateur connecté (private-chat.user.{personId}) et maintient la liste de toutes les conversations avec leurs compteurs de messages non lus. Polling toutes les 8 secondes comme fallback quand Reverb est indisponible. Écoute également le CustomEvent covoit:chat-read-changed pour forcer un re-rendu quand l'état non-lu change en localStorage.

Clients Http – Axios

6.1 apiClient — client interne (src/app/apiClient.ts)

Client Axios configuré avec withCredentials: true pour transmettre automatiquement le cookie de session httpOnly à chaque requête vers le backend Laravel.

```
export const apiClient = axios.create({
  baseURL: API_BASE_URL,
  headers: { "Content-Type": "application/json", Accept: "application/json" },
  withCredentials: true,  // cookie httpOnly transmis automatiquement
});
```

Intercepteur de requête : injecte l'en-tête Accept-Language avec la locale active (getCurrentLocale()) à chaque appel.

Intercepteur de réponse 401 : si la requête n'appartient pas à la liste des chemins auth exclus (/auth/login, /auth/me, /auth/refresh…) et qu'elle n'a pas encore été retentée (_retry), tente POST /auth/refresh puis rejoue la requête originale. Si le refresh échoue, l'erreur est propagée.

```
// Logique de retry après 401
if (error.response?.status === 401 && !originalRequest._retry) {
  originalRequest._retry = true;
  await axios.post(`${API_BASE_URL}/auth/refresh`, {}, { withCredentials: true });
  return apiClient(originalRequest); // rejoue la requête originale
}
```

6.2 externalApiClient — services tiers (src/app/externalApiClient.ts)

Client Axios sans baseURL, sans withCredentials. Utilisé exclusivement pour les services publics tiers :

- geo.api.gouv.fr — recherche de communes françaises par nom (autocomplétion d'adresse)
- data.geopf.fr — géocodage d'adresse textuelle en coordonnées GPS (latitude / longitude)

6.3 Gestion des erreurs API (src/app/apiError.ts)

extractApiErrorMessage(err) inspecte la réponse HTTP et retourne le message d'erreur le plus précis en suivant cette priorité :

1. Premier message dans data.fields (erreurs de validation par champ — erreurs Laravel 422)
2. data.details (message détaillé)
3. data.message (message générique)
4. err.message (message natif Axios)

```
// src/app/apiError.ts
export function extractApiErrorMessage(err: unknown): string {
  if (!axios.isAxiosError(err)) return err instanceof Error ? err.message : "Unexpected
error";
  const data = err.response?.data as ApiErrorPayload;
  const firstFieldMessage = fields
    ? Object.values(fields).find(msgs => msgs.length > 0)?.[0] : null;
  return firstFieldMessage ?? details ?? message ?? err.message ?? "Request failed";
}
```

Authentification et gestion de session

7.1 Stratégie d'authentification — JWT custom (firebase/php-jwt)

L'API n'utilise pas Laravel Sanctum. Elle implémente un système d'authentification JWT entièrement personnalisé basé sur la bibliothèque firebase/php-jwt (algorithme HS256). La session repose sur deux tokens httpOnly distincts délivrés par le backend :

Token	Type	Cookie	TTL par défaut	Stockage backend
Access token	JWT signé HS256	access_token (httpOnly)	15 minutes (JWT_ACCESS_TTL=900)	Aucun — vérifié par signature
Refresh token	Opaque (random_bytes(32))	refresh_token (httpOnly)	30 jours (JWT_REFRESH_TTL=2592000)	Table refresh_tokens (hash SHA-256)

Les deux cookies sont httpOnly, inaccessibles en JavaScript, protégeant les tokens contre les attaques XSS. La configuration JWT est définie dans config/jwt.php : JWT_SECRET (clé de signature), JWT_ISSUER, JWT_AUDIENCE.

- POST /auth/login : ouvre la session et definit les cookies, puis renvoie uniquement un message de succes.
- POST /auth/register : cree le compte, definit les cookies de session, puis renvoie uniquement un message de succes.
- POST /auth/refresh : renouvelle la session a partir du cookie de refresh, puis renvoie uniquement un message de succes.
- Le frontend restaure l'utilisateur via GET /auth/me apres login ou inscription.

7.2 Payload JWT (access token)

Le payload du token JWT access contient les claims standards RFC 7519 plus des claims applicatifs :

```
// Payload JWT signé HS256
{
  "iss": "couvoit-api",          // émetteur (JWT_ISSUER)
  "aud": "couvoit-client",        // audience (JWT_AUDIENCE)
  "iat": 1711367400,              // horodatage d'émission
  "exp": 1711368300,              // expiration (iat + access_ttl)
  "sub": "42",                    // user id (string)
  "email": "user@example.com",
  "role_id": 1,
  "jti": "a3f7b9..."              // identifiant unique du token
}
```

7.3 Middleware LocalJwtAuth — Authentification des requêtes

Le middleware LocalJwtAuth (app/Http/Middleware/LocalJwtAuth.php) authentifie chaque requête protégée via le middleware jwt. Il implémente une logique de cache Redis pour éviter les

requêtes DB répétées :

5. Lit l'access token depuis le cookie access_token (priorité) ou l'en-tête Authorization: Bearer (fallback)
6. Vérifie la signature HMAC-SHA256 + les claims iss, aud, sub via JwtIssuer::verify()
7. Calcule un fingerprint SHA-256 du token entier

8. Vérifie le cache Redis (clé local:auth:{sub}) : si présent et fingerprint identique → résout l'utilisateur depuis le cache
9. Sinon : charge l'utilisateur depuis la base, stocke en cache avec TTL aligné sur l'expiration du token
10. Vérifie que l'utilisateur est actif (is_active = true)
11. Injecte l'utilisateur via auth()->guard()->setUser($user)

```
// app/Http/Middleware/LocalJwtAuth.php — logique de cache
$authCacheKey = "local:auth:" . $sub;
$cached = Cache::get($authCacheKey);

if (hash_equals($cached["token_fp"], $tokenFp)) {
    // Cache hit — pas de requête DB
    $user = User::find($cached["user_id"]);
} else {
    // Cache miss ou fingerprint différent — charge depuis DB
    Cache::put($authCacheKey, ["token_fp" => $tokenFp, "user_id" => $user->id], $ttl);
}
```

7.4 Rotation des refresh tokens

Les refresh tokens sont opaques (bin2hex(random_bytes(32))) et stockés hashés (SHA-256) dans la table refresh_tokens. À chaque rafraîchissement, l'ancien token est révoqué (revoked_at) et un nouveau token est émis atomiquement (consumeAndRotate). Cette rotation empêche la réutilisation d'un token compromis.

```
// RefreshRequest::prepareForValidation() — lit le cookie automatiquement
protected function prepareForValidation(): void {
    $this->merge([
        "refresh_token" => $this->cookie("refresh_token"),
    ]);
}

// AuthService::refresh() — rotation atomique
$userId = $this->refreshTokens->consumeAndRotate(
    $refreshToken,          // révoque l'ancien
    $newRefresh,            // stocke le nouveau
    $expiresAt              // TTL 30 jours
);
$access = $this->jwt->issueAccessToken($user);
// Retourne les deux cookies httpOnly mis à jour
```

7.5 Côté frontend — cycle de rafraîchissement transparent

Côté frontend (src/app/apiClient.ts), l'intercepteur de réponse 401 déclenche automatiquement POST /auth/refresh. Le backend lit le cookie refresh_token directement via prepareForValidation(),

sans que le frontend n'ait à extraire ou transmettre le token manuellement. Les nouveaux cookies sont définis par le Set-Cookie de la réponse HTTP.

```
// src/app/apiClient.ts — intercepteur 401
if (error.response?.status === 401 && !originalRequest._retry) {
    originalRequest._retry = true;
    // POST /auth/refresh — le cookie refresh_token est envoyé automatiquement
    await axios.post(`${API_BASE_URL}/auth/refresh`, {}, { withCredentials: true });
    return apiClient(originalRequest); // rejoue la requête avec le nouveau cookie
}
```

7.6 Modèle AuthUser

L'objet AuthUser est résolu depuis GET /auth/me et mis en cache dans AuthContext. Il contient l'identité de l'utilisateur, son rôle et ses permissions granulaires calculées à la volée par le backend.

```
// src/types/MeResponse.ts
export interface AuthUser {
  id: number;
  email: string;
  is_active: boolean;
  role: { id: number; name: "admin" | "driver" | "passenger" };
  person: AuthPerson | null;     // null tant que le profil n'est pas complété
  permissions: AuthPermissions;
}

export interface AuthPermissions {
  can_view_bookings: boolean;
  can_book_trip: boolean;
  can_cancel_booking: boolean;
  can_edit_profile: boolean;
  can_publish_trip: boolean;
  can_manage_own_trips: boolean;
  can_manage_all_trips: boolean;
  can_manage_all_users: boolean;
  can_manage_all_bookings: boolean;
}
```

7.7 Complétion de profil (src/auth/profileCompletion.ts)

isProfileComplete(user) vérifie que les champs first_name, last_name (minimum 2 caractères) et pseudo (minimum 3 caractères) sont renseignés sur l'objet person. Les routes protégées redirigent vers /complete-profile si ce test échoue.

```
// src/auth/profileCompletion.ts
export function isProfileComplete(user: AuthUser | null): boolean {
  const person = user?.person;
  if (!person) return false;
```

```
  return hasText(person.first_name, 2)
      && hasText(person.last_name, 2)
      && hasText(person.pseudo, 3);
}
```

Routage et gardes de routes

Le routage est déclaré dans src/router/AppRouter.tsx avec React Router v7. Les guards sont des composants qui wrappent des groupes de routes via le pattern Outlet.

Guard	Comportement	Redirection si échec
GuestRoute	Redirige les utilisateurs authentifiés hors des pages auth	/home
ProtectedRoute	Requiert session valide + profil complet	/login ou /complete-profile
PermissionRoute	Requiert une permission backend spécifique	/home
AdminRoute	Requiert role=admin ou can_manage_all_users	/login

8.1 ProtectedRoute — logique de décision

ProtectedRoute évalue les conditions dans cet ordre strict :

12. status === "loading" → écran de chargement (spinner centré, bloque le rendu)
13. status === "guest" → redirect /login avec state.flash (message FloatingToast affiché sur LoginPage)
14. isProfileComplete(user) === false → redirect /complete-profile
15. user.role.name === "admin" ou can_manage_all_users → redirect /admin
16. Cas nominal → rendu de <AppLayout /> (contient <Outlet />)

8.2 PermissionRoute

Vérifie user.permissions[permission] === true. Si la permission est absente, redirige vers /home. Les permissions sont des booléens accordés par le backend selon le rôle de l'utilisateur.

```
// src/router/PermissionRoute.tsx
if (!user?.permissions[permission]) {
  return <Navigate to="/home" replace />;
}
```

8.3 Table complète des routes

Chemin	Guard	Page
/	Aucun	LandingPage
/login	GuestRoute	LoginPage
/register	GuestRoute	RegisterPage
/forgot-password	GuestRoute	ForgotPasswordPage
/reset-password	GuestRoute	ResetPasswordPage
/complete-profile	Aucun	CompleteProfilePage
/home	ProtectedRoute	HomePage
/find-trip	ProtectedRoute	FindTripPage
/find-trip/results	ProtectedRoute	TripResultsPage
/trips/:tripId	ProtectedRoute	TripDetailsPage
/trips/:tripId/contact-driver	ProtectedRoute	ContactDriverPage
/trips/:tripId/contact-driver-email	ProtectedRoute	ContactDriverEmailPage
/chat	ProtectedRoute	ChatInboxPage
/chat/:conversationId	ProtectedRoute	ChatConversationPage
/support	ProtectedRoute	SupportPage
/my-trips	PermissionRoute (can_manage_own_trips)	MyTripsPage
/my-trips/new	PermissionRoute (can_manage_own_trips)	PublishTripPage
/my-trips/:tripId	PermissionRoute (can_manage_own_trips)	DriverTripDetailsPage
/my-trips/:tripId/contact-passenger/:id	PermissionRoute	ContactPassengerPage
/bookings	PermissionRoute (can_view_bookings)	MyBookingsPage
/bookings/:tripId	PermissionRoute (can_view_bookings)	BookingDetailsPage
/my-account	PermissionRoute (can_edit_profile)	MyAccountPage
/admin	AdminRoute + AdminLayout	AdminDashboardPage
/admin/users	AdminRoute	AdminUsersPage
/admin/trips	AdminRoute	AdminTripsPage
/admin/brands	AdminRoute	AdminBrandsPage
/admin/models	AdminRoute	AdminModelsPage
/admin/cars	AdminRoute	AdminCarsPage
/admin/support	AdminRoute	AdminSupportInboxPage
/admin/support/:sessionId	AdminRoute	AdminSupportChatPage
*	Aucun	Redirect → /

Internationalisation (i18n)

L'application implémente son propre système d'internationalisation sans dépendance externe. Le module est centré autour de src/i18n/config.ts et src/i18n/dictionaries.ts.

9.1 Locales supportées

Code	Langue	Direction	Intl locale
en	Anglais (défaut)	LTR	en-US
fr	Français	LTR	fr-FR
ar	Arabe	RTL — document.dir = "rtl"	ar

9.2 Résolution de la locale initiale

resolveInitialLocale() suit cette priorité au chargement du module :

17. localStorage clé covoit.language (choix précédent de l'utilisateur)
18. Préférences du navigateur (window.navigator.languages — première locale supportée)
19. "en" par défaut si aucune correspondance

La locale est appliquée immédiatement au chargement du module i18n (avant le premier rendu React) via applyLocaleToDocument() : document.documentElement.lang = locale et document.documentElement.dir = direction.

9.3 Fonction translate()

```
// src/i18n/config.ts
export function translate(
  key: string,
  values?: Record<string, string | number>,
  locale: AppLocale = getCurrentLocale()
): string {
  const template = dictionaries[locale][key]    // locale active
    ?? dictionaries.en[key]                      // fallback anglais
    ?? key;                                       // fallback = clé brute

  if (!values) return template;

  // Remplacement des tokens {token} par leurs valeurs
  return Object.entries(values).reduce(
    (msg, [token, val]) => msg.replaceAll(`{${token}}`, String(val)),
    template
  );
}
```

Exemple d'utilisation avec valeurs dynamiques :

```
// dictionaries.ts
// "trip.seats": "Il reste {count} place(s) disponible(s)"

t("trip.seats", { count: 3 })
// → "Il reste 3 place(s) disponible(s)"
```

9.4 Hook useI18n et I18nProvider

I18nProvider est un contexte React qui expose la locale active, la fonction t() (alias de translate()), et setLocale(). Tous les composants utilisent le hook useI18n() plutôt que translate() directement pour s'assurer de réagir aux changements de locale. Le composant LanguageSwitcher utilise setLocale() pour changer la langue à la volée.

9.5 Formatage des dates

formatLocaleDateTime() et formatLocaleTime() utilisent l'API native Intl.DateTimeFormat pour formater selon la locale active. Les formats diffèrent par locale :

```
// Rendu pour une même date selon la locale active
formatLocaleDateTime("2026-03-25T14:30:00Z")
// EN → "Mar 25, 2026, 2:30 PM"
// FR → "25 mars 2026, 14:30"
// AR → "٢٥ مارس ٢٠٢٦، ٢:٣٠ م"
```

Chat en temps réel — WebSocket

Le système de messagerie repose sur Laravel Echo avec le broadcaster Reverb (compatible protocole Pusher). Il couvre deux niveaux de canaux : le canal inbox utilisateur (notifications inter-conversations) et le canal de conversation active (messages en direct).

10.1 Initialisation du client Echo (chatEcho.ts)

getChatEcho() retourne un singleton Echo configuré pour Reverb. Il utilise un authorizer custom qui passe par apiClient (avec cookie httpOnly) pour authentifier les canaux privés, au lieu d'utiliser un token dans l'URL WebSocket.

```
// src/features/chat/chatEcho.ts
echoInstance = new Echo({
  broadcaster: "reverb",
  key: import.meta.env.VITE_REVERB_APP_KEY,
  wsHost: resolveHost(),          // VITE_REVERB_HOST
  wsPort: resolvePort(false),     // VITE_REVERB_WS_PORT
  wssPort: resolvePort(true),     // VITE_REVERB_WSS_PORT
  forceTLS: forceTls,             // basé sur VITE_REVERB_SCHEME
  enabledTransports: ["ws", "wss"],
  authorizer:  (channel)  =>  ({
    authorize: (socketId, callback) => {
      apiClient
        .post("/broadcasting/auth-proxy", {
          socket_id: socketId,
          channel_name: channel.name,
        })
        .then(res => callback(null, res.data))    // succès
        .catch(err => callback(err, null));         // échec
    }
  })
});
```

10.2 useChatRealtime

Hook générique (src/features/chat/useChatRealtime.ts) qui prend un channelName et un callback onSignal. S'abonne au canal privé et écoute l'événement .chat.message.sent. Expose isRealtimeConnected (booléen). Nettoie automatiquement au démontage (stopListening + leaveChannel).

10.3 Table des canaux WebSocket

Canal	Abonné par	Événement écouté	Déclencheur
private-chat.user.{personId}	ChatInboxProvider	.chat.message.sent	Nouveau message dans n'importe quelle conversation
private-chat.conversation.{id}	useChatConversation	.chat.message.sent	Nouveau message dans cette conversation précise
private-chat.support.session.{id}	useSupportChatRealtime	.chat.message.sent	Message de support admin
private-admin.support.inbox	useAdminSupportRealtime	admin.support.*	Nouvelle session ou message entrant (admin)

10.4 État non-lu (chatReadState.ts)

Persiste dans localStorage sous covoit.chat.read-state une map indexée par id de conversation. Chaque entrée contient readAt (horodatage de la dernière lecture), readMessageId (id du dernier message lu) et unreadCount (compteur d'incréments).

Fonction	Comportement
markConversationRead(id, ts, msgId)	Remet unreadCount à 0, enregistre readAt et readMessageId
incrementConversationUnread(id)	Incrémente unreadCount de 1 (appel lors d'un message entrant)
syncConversationUnread(id, updatedAt, msgId, sender)	Compare l'état local au dernier message — si sender="other" et plus récent que readAt/readMessageId : marque non-lu
getConversationUnread(id, …)	Retourne true si unreadCount > 0 ou si le message le plus récent est plus récent que la dernière lecture

Chaque écriture en localStorage déclenche un CustomEvent covoit:chat-read-changed. ChatInboxProvider écoute cet événement pour forcer un re-rendu et mettre à jour le badge de navigation.

10.5 useChatConversation — hook de conversation

Ce hook combine plusieurs mécanismes pour garantir la fraîcheur des messages :

- Chargement initial : GET /conversations/{id} avec messages[]
- Polling silencieux toutes les 5 secondes (fallback si WebSocket indisponible)
- Abonnement WebSocket au canal private-chat.conversation.{id}
- handleSubmit : POST /conversations/{id}/messages avec FormData (texte + fichiers)
- Mise à jour optimiste côté expéditeur (avant l'écho WebSocket)
- handleClearConversation / handleClearMessages : suppression de vue locale uniquement

```
// src/hooks/Chat/useChatConversation.ts

// Polling fallback 5s
const interval = window.setInterval(() => void load(true), 5000);

// Abonnement WebSocket
useChatRealtime(
  `chat.conversation.${conversationId}`,
  () => void load(true)            // déclenche un reload silencieux
);

// Mise à jour optimiste à l'envoi
const created = await sendConversationMessage(id, content, files);
setConversation(prev => ({
  ...prev,
  messages: [...prev.messages, created],   // visible immédiatement
  latestMessage: created,
}));
```

Support en direct

Le module de support permet à un utilisateur authentifié d'ouvrir une session de chat avec un agent administrateur. Il couvre deux interfaces : le widget flottant côté utilisateur et l'interface de gestion côté administrateur.

11.1 Côté utilisateur — SupportChatWidget

Le widget est un panneau flottant à position fixe. Sur mobile, il est positionné à bottom-20 (80px du bas) pour passer au-dessus de la barre de navigation inférieure et s'étend en plein écran (inset-x-2, h-[calc(100dvh-5.5rem)]) en utilisant dvh (dynamic viewport height) pour gérer correctement le clavier virtuel. Sur desktop (sm+), il est limité à 380px de large avec sm:bottom-6.

Le hook useSupportChat gère le cycle de vie complet de la session :

- Création automatique de session à l'ouverture du widget : POST /support/sessions
- Abonnement WebSocket au canal private-chat.support.session.{id}
- Indicateur de frappe administrateur (isAdminTyping)
- Indicateur de statut de connexion (connecting / connected / disconnected)
- Fermeture propre de la session à la destruction du hook

11.2 Côté administrateur — Interface de gestion

AdminSupportInboxPage liste les sessions en attente (waiting) et actives (active), filtrées par onglet. AdminSupportChatPage ouvre une session spécifique avec toutes les fonctionnalités d'un agent : réponse, envoi de pièces jointes, clôture de session. La hauteur du conteneur de chat est adaptée à l'écran :

```
// Hauteur corrigée pour mobile et desktop
// Mobile  : header 56px + padding 32px = 88px = 5.5rem
// Desktop : header 64px + padding 64px = 128px = 8rem
className="h-[calc(100vh-5.5rem)] lg:h-[calc(100vh-8rem)]"
```

useAdminSupportRealtime s'abonne au canal private-admin.support.inbox pour recevoir les nouvelles sessions et messages entrants en temps réel, sans nécessiter de polling.

Gestion du chargement global

12.1 Mécanisme de chargement global basé sur un compteur

Le système de chargement global utilise un compteur d'entiers plutôt qu'un booléen pour gérer correctement les requêtes parallèles sans faux positifs.

```
// Comportement du compteur avec deux requêtes parallèles
// Requête A démarre → pendingRequests = 1 → isLoading = true  ✓
// Requête B démarre → pendingRequests = 2
// Requête A termine → pendingRequests = 1 → isLoading = true  ✓ (B encore active)
// Requête B termine → pendingRequests = 0 → isLoading = false ✓
```

Les appels silencieux (polling chat, inbox) passent showGlobalLoader: false dans leur config Axios pour ne pas déclencher le spinner. Ce flag est défini par une augmentation de type dans src/types/axios.d.ts :

```
// src/types/axios.d.ts
declare module "axios" {
  interface AxiosRequestConfig {
    showGlobalLoader?: boolean;
  }
}

// Usage
await apiClient.get("/conversations", { showGlobalLoader: false });
```

Gestion des erreurs globales

L'application distingue deux niveaux d'erreurs avec des mécanismes différents :

Mécanisme	Portée	Composant d'affichage
ErrorContext + GlobalErrorAlert	Erreurs critiques globales (réseau, crash)	Bannière fixe en haut de page
état error dans le hook	Erreurs métier locales (formulaire, envoi)	FloatingToast ou message inline
location.state.flash	Message de redirection entre pages	FloatingToast sur la page cible

13.1 Flash message de redirection

Lorsque ProtectedRoute redirige un utilisateur non connecté vers /login, elle passe un message via React Router state. LoginPage lit ce message et affiche un FloatingToast d'erreur :

```
// ProtectedRoute.tsx — passage du message
return <Navigate to="/login" replace
  state={{ from: location, flash: t("auth.signInRequired") }} />;

// LoginPage.tsx — lecture et affichage
const flash = (location.state as { flash?: string } | null)?.flash ?? null;
return (
  <>
    <FloatingToast message={flash} tone="error" durationMs={5000} />
    <LoginSection ... />
  </>
);
```

Modèles de données (Types TypeScript)

14.1 ApiResponse<T> — enveloppe standard

```
// src/types/ApiResponse.ts
export interface ApiResponse<T> {
  data: T;
}
```

14.2 AuthUser et AuthPermissions

Définis dans src/types/MeResponse.ts. AuthUser est l'objet utilisateur résolu depuis /auth/me. Les permissions sont des booléens individuels accordés par le backend selon le rôle assigné à l'utilisateur.

14.3 ChatConversation et ChatMessage

```
// src/types/Chat.ts — formes frontend normalisées
interface ChatConversation {
  id: number;
  participantId: number;        // ID de l'autre participant
  participantName: string;
  participantSubtitle: string;  // "@pseudo" si disponible
  tripId: number;
  tripLabel: string;            // "Paris - Lyon"
  updatedAt: string;
  clearedAt: string | null;     // si l'utilisateur a effacé sa vue locale
  latestMessage: ChatMessage | null;
  messages: ChatMessage[];
}

interface ChatMessage {
  id: number;
  body: string;
  sender: "me" | "other";       // relatif à l'utilisateur courant
  senderPersonId: number;
  createdAt: string;
  attachments: ChatAttachment[];
}
```

Les types API bruts (ChatMessageApi, ChatConversationApi) sont définis localement dans chatApi.ts et transformés par des fonctions mapMessage() et mapConversation() avant d'être exposés aux composants.

14.4 Trip

Défini dans src/types/Trip.ts. Contient les informations complètes d'un trajet : identifiant, ville d'origine, ville de destination, horaire de départ, prix, nombre de places disponibles, statut de réservation de l'utilisateur courant, informations du conducteur et du véhicule.

14.5 Address, Commune, GeoPfSearchResponse

Types dédiés aux résultats de géocodage. Commune (src/types/Commune.ts) représente une commune française retournée par geo.api.gouv.fr. GeoPfSearchResponse (src/types/GeoPfSearchResponse.ts) représente le résultat de géocodage d'adresse par data.geopf.fr.

Composants principaux

15.1 AppLayout (src/components/layout/AppLayout.tsx)

Shell principal de l'application authentifiée. Contient le TopNav (desktop), le BottomNav (mobile), un <Outlet /> pour le rendu des pages, et un footer. Gère la déconnexion via logoutLocal() depuis AuthContext.

15.2 AdminLayout (src/components/layout/AdminLayout.tsx)

Shell de l'interface d'administration. Sidebar collapsible sur mobile (overlay semi-transparent + bouton hamburger). Sur desktop (lg+), la sidebar est affichée en permanence. En-tête fixe avec LanguageSwitcher, lien retour vers l'app et bouton logout.

15.3 BottomNav (src/components/layout/BottomNav.tsx)

Barre de navigation inférieure visible uniquement sur mobile (lg:hidden). Filtre dynamiquement les onglets selon les permissions de l'utilisateur connecté (propriété mobileVisible de chaque navItem). Affiche un badge rouge animé sur l'onglet Chat quand unreadCount > 0. Positionné à fixed bottom-0, z-50 — tous les éléments flottants (SupportChatTrigger, SupportChatWidget) sont ajustés à bottom-20 pour ne pas être masqués.

15.4 SerenePrimitives (src/components/common/SerenePrimitives.tsx)

Bibliothèque de primitives React réutilisables. Applique les classes CSS sémantiques définies dans src/index.css : serene-card, serene-button-primary, serene-button-secondary, serene-page-shell, serene-nav-link, serene-input, etc. Ces tokens permettent un theming cohérent sans répéter les classes Tailwind.

15.5 FloatingToast (src/components/common/FloatingToast.tsx)

Toast animé à position fixe. Rendu null quand message est null ou vide. Se retire automatiquement après durationMs millisecondes. Props : message (string | null), tone ("success" | "error"), durationMs.

15.6 GlobalSpinner (src/components/common/GlobalSpinner.tsx)

Overlay plein écran semi-transparent avec spinner centré. Rendu uniquement quand isLoading === true (LoadingContext). Invisible pour les appels avec showGlobalLoader: false.

15.7 LandingNavbar (src/components/common/LandingNavbar.tsx)

Barre de navigation de la page d'accueil publique. Adapte ses liens selon l'état d'authentification : boutons "Se connecter" / "S'inscrire" pour les visiteurs, lien vers /home ou /complete-profile pour les utilisateurs connectés.

Conventions et patterns de développement

16.1 Pattern pages fines (thin pages)

Les pages dans src/pages/ sont intentionnellement minimalistes : elles importent un hook et transmettent les données à un composant section. Toute la logique (état, appels API, handlers) vit dans le hook.

```
// Pattern standard : page fine
export default function MyTripsPage() {
  const { trips, loading, error, handlePublish } = useMyTrips();
  return (
    <MyTripsSection
      trips={trips}
      loading={loading}
      error={error}
      onPublish={handlePublish}
    />
  );
}
```

16.2 Hooks propriétaires

Les hooks dans src/hooks/ possèdent tout l'état local et orchestrent les appels API via les fonctions de src/features/. Ce découplage permet de :

- Tester les fonctions API indépendamment des composants React
- Réutiliser un hook dans plusieurs pages sans dupliquer la logique
- Remplacer une implémentation API sans toucher aux composants UI

16.3 Pas de gestion d'état globale externe

L'application n'utilise ni Redux, ni Zustand, ni React Query. L'état vit dans les hooks locaux et les contextes React. ChatInboxProvider et AuthProvider sont les seuls états vraiment globaux. Ce choix réduit la complexité et les dépendances.

16.4 Gestion des erreurs dans les hooks

Chaque hook maintient son propre état error (string | null) et success (string | null). Les erreurs sont extraites via extractApiErrorMessage(). Elles sont affichées localement via FloatingToast ou un champ d'erreur inline — elles ne remontent pas dans GlobalErrorAlert.

16.5 Styling — Tailwind + tokens CSS

Tailwind CSS est utilisé via PostCSS. Les tokens sémantiques globaux sont définis dans src/index.css : --theme-primary, --theme-ink, --theme-surface, --theme-line, --theme-muted, --theme-coral, --theme-bg-soft, etc. Les composants utilisent ces variables CSS plutôt que des couleurs Tailwind directes pour garantir la cohérence visuelle et faciliter un éventuel theming.

16.6 FormData pour les pièces jointes

Tous les endpoints qui acceptent des fichiers utilisent multipart/form-data via l'API FormData native de JavaScript. Les pièces jointes sont limitées à 5 fichiers maximum côté frontend (slice(0, 5) lors de la sélection).

16.7 Mises à jour optimistes

À l'envoi d'un message chat, le message est ajouté immédiatement à l'état local via la réponse HTTP (201), sans attendre l'écho WebSocket. Cela garantit une expérience utilisateur réactive même avec une latence réseau.

Sécurité

17.1 Aucun token sensible en stockage client

Suite à un audit de sécurité, tous les tokens d'authentification et identifiants sensibles ont été retirés du localStorage et du sessionStorage. Les tokens JWT (access et refresh) sont transmis exclusivement via cookies httpOnly définis par le backend — inaccessibles en JavaScript, protégés contre les attaques XSS.

Donnée	Stockage actuel	Raison
access_token (JWT)	Cookie httpOnly (backend Laravel)	Inaccessible JS — protégé XSS
refresh_token (opaque)	Cookie httpOnly (backend Laravel)	Inaccessible JS — rotation à chaque usage
personId	AuthContext en mémoire	Résolu depuis user?.person?.id
Locale	localStorage (covoit.language)	Non-sensible — préférence UX
État non-lu chat	localStorage (covoit.chat.read-state)	Non-sensible — confort UX

17.2 Authentification des canaux WebSocket privés

L'authorizer Echo custom passe par apiClient (avec cookie) pour signer les abonnements aux canaux privés via /broadcasting/auth-proxy. Cela évite l'exposition d'un token dans l'URL WebSocket ou dans les en-têtes Pusher. Le backend vérifie la session Laravel avant de retourner la signature du canal.

17.3 Guards de routes côté frontend

Les guards (ProtectedRoute, PermissionRoute, AdminRoute) offrent une protection d'expérience utilisateur mais ne remplacent pas la sécurité backend. Chaque endpoint Laravel vérifie indépendamment les droits. Le frontend ne fait que refléter ce que le backend autorise.

17.4 Interception transparente des 401

L'intercepteur de réponse dans apiClient.ts intercepte silencieusement les HTTP 401 et tente un refresh transparent avant de propager l'erreur. Si le refresh échoue, AuthProvider met le statut à "guest" et l'utilisateur est redirigé vers /login avec un message explicatif.

17.5 Protection contre les clones d'objets (DataCloneError)

React Router navigate() et Navigate sérialisent le state via l'API History. Il est essentiel de passer un objet plain sérialisable dans state, notamment l'objet location de React Router (useLocation()) plutôt que window.location (un objet natif non cloneable).

```
// CORRECT — objet sérialisable de React Router
const location = useLocation();
return <Navigate to="/login" state={{ from: location, flash: "..." }} />;

// INCORRECT — provoque DataCloneError
// state={{ from: window.location }}
```